

M5_Practice

Danni Martin

2024-11-25

Time Series II - Benchmarks

Benchmarks serve as a baseline/reference point when comparing performance of other models.

Three Principles of Forecasting:

1. All forecasts are wrong, though some are useful.
2. The longer the forecast, the more wrong it is.
3. Aggregate forecasts tend to be more accurate.

Benchmarks

1. **Naive** - value of future period will equal value of preceding period. ie uses previous observation to predict the future. `model(NAIVE(data))`
2. **Drift** - variant of Naive method. Forecasts increase or decrease depending on the average change seen in historical data. `model(RW(data ~ drift()))`. RW = Random Walk
3. **Mean** - future values are equal to the average of the historical data. `model(MEAN(data))`
4. **Least Squares** - Type of linear regression analysis based on minimizing the sum of squares of the residuals. `model(TLSM())`

Note

Use `model()` function to run benchmarks. Object will be saved as a mable. (model table)

Accuracy Measures

1. **MSE** - measures the accuracy of model relative to observed values.
2. **RMSE** - square root of MSE. Used more often because it is measured in the same units of the response variable.
3. **AIC** - penalizes model complexity and rewards fit. Balance between model fit and number of parameters
4. **AICc** - AIC adjustment for smaller sample sizes
5. **BIC** - penalizes model complexity to a greater degree than AIC. Based on Bayesian principles.

Cross Validation

Cross Validation is a method to avoid overfitting. Overfitting occurs when the model is too flexible; therefore, model fitted to noise instead of underlying pattern.

Average Price of Avocados

Modeling

```
# Load the data.  
library(tidyverse) #data wrangling
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --  
v dplyr      1.1.4      v readr      2.1.5  
v forcats    1.0.0      v stringr    1.5.1  
v ggplot2    3.5.1      v tibble     3.2.1  
v lubridate  1.9.3      v tidyr      1.3.1  
v purrr      1.0.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
```

```
x dplyr::filter() masks stats::filter()
```

```
x dplyr::lag()     masks stats::lag()
```

```
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(fpp3) #handle time series data
```

Warning: package 'fpp3' was built under R version 4.4.2

Registered S3 method overwritten by 'tsibble':

```
  method          from
  as_tibble.grouped_df dplyr
-- Attaching packages ----- fpp3 1.0.1 --
v tsibble      1.1.5      v feasts      0.4.1
v tsibbledata  0.4.1      v fable      0.4.1
```

Warning: package 'tsibble' was built under R version 4.4.2

Warning: package 'tsibbledata' was built under R version 4.4.2

Warning: package 'feasts' was built under R version 4.4.2

Warning: package 'fabletools' was built under R version 4.4.2

Warning: package 'fable' was built under R version 4.4.2

```
-- Conflicts ----- fpp3_conflicts --
x lubridate::date()      masks base::date()
x dplyr::filter()        masks stats::filter()
x tsibble::intersect()   masks base::intersect()
x tsibble::interval()    masks lubridate::interval()
x dplyr::lag()           masks stats::lag()
x tsibble::setdiff()     masks base::setdiff()
x tsibble::union()       masks base::union()
```

```
cali<-read_csv("https://jagelves.github.io/Data/CaliforniaAvocado.csv")
```

Rows: 205 Columns: 4

```
-- Column specification -----
Delimiter: ","
chr  (1): geography
dbl  (2): average_price, total_volume
date (1): date
```

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

Model Coefficients For The Avocado Data

.model	term	estimate	std.error	statistic	p.value
mean	mean	1.6845	0.0212	79.3980	0.0000
Drift	b	0.0021	0.0104	0.2010	0.8410
LS	(Intercept)	-3.7322	0.9230	-4.0438	0.0001
LS	date	0.0003	0.0001	5.8702	0.0000

```
# Coerce data into tsibble. Select date and average price. Set index to date.
# Filter to specified date range.
```

```
cali %>%
  select(date,average_price) %>%
  as_tsibble(index=date) %>%
  filter_index("2015-01-04"~"2018-06-02") -> calits_train
```

```
# Use model function to apply four benchmarks.
fit <- model(calits_train,mean=MEAN(average_price),
             Naive=NAIVE(average_price),
             Drift=RW(average_price~drift()),
             LS=TSLM(average_price~date))
```

```
# Use coef() to explore coefficients of model. Format using gt package.
library(gt) #customize tables
```

Warning: package 'gt' was built under R version 4.4.2

```
coef(fit) %>%
  gt() %>%
  cols_align("center") %>%
  tab_header(title = md("**Model Coefficients For The Avocado Data**")) %>%
  tab_style(
    locations =cells_column_labels(columns = everything()),
    style = list(cell_borders(sides = "bottom", weight = px(3)),
                 cell_text(weight = "bold"))) %>%
  fmt_number(columns =c(statistic,estimate,std.error,p.value),
             decimals = 4)
```

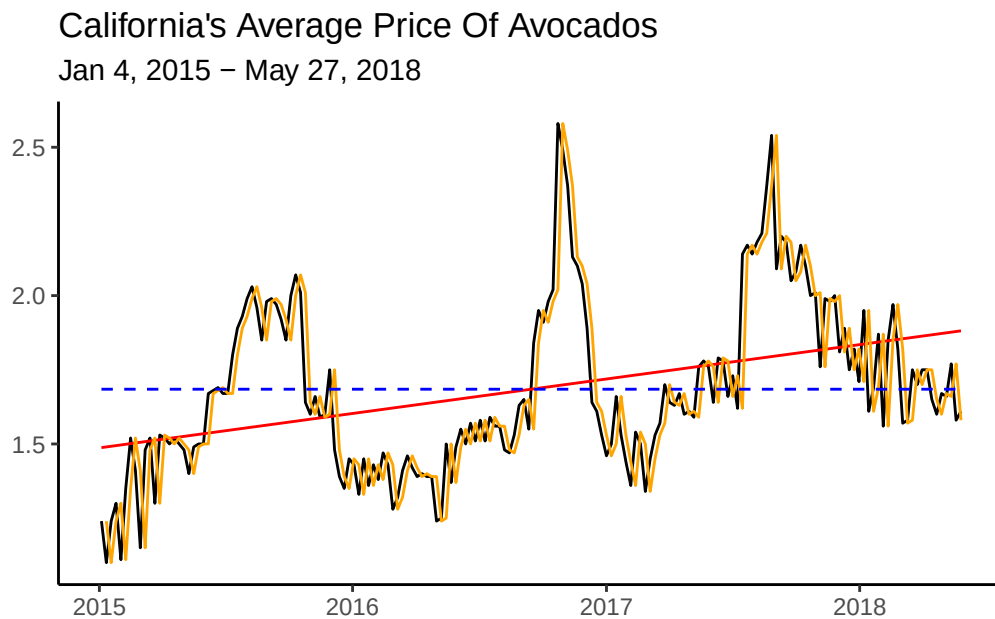
```

# Visualize.
# Red - Least Squares
# Blue - Mean
# Orange - Naive

calits_train %>% autoplot(average_price) + theme_classic() +
  geom_line(aes(y = .fitted), col="red",
            data = augment(fit) %>% filter(`.model`=="LS")) +
  geom_line(aes(y = .fitted), col="orange",
            data = augment(fit) %>% filter(`.model`=="Naive")) +
  geom_line(aes(y = .fitted), col="blue", linetype="dashed",
            data = augment(fit) %>% filter(`.model`=="mean")) +
  labs(y="", title= "California's Average Price Of Avocados",
       subtitle = "Jan 4, 2015 - May 27, 2018",
       x="")

```

Warning: Removed 1 row containing missing values or values outside the scale range (``geom_line()``).



Model Fit

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
mean	Training	0.00	0.28	0.23	-2.71	13.59	2.26	2.05	0.87
Naive	Training	0.00	0.14	0.10	-0.19	6.04	1.00	1.00	-0.21
Drift	Training	0.00	0.14	0.10	-0.31	6.05	1.00	1.00	-0.21
LS	Training	0.00	0.26	0.21	-2.21	12.23	2.07	1.87	0.85

Model Fit

Model fit is assessed by comparing fitted values against observed values. Smaller accuracy values equates to higher accuracy.

Mean Squared Error (MSE) - average squared difference between predicted values and actual values, i.e. average squared error. Used for assessing accuracy.

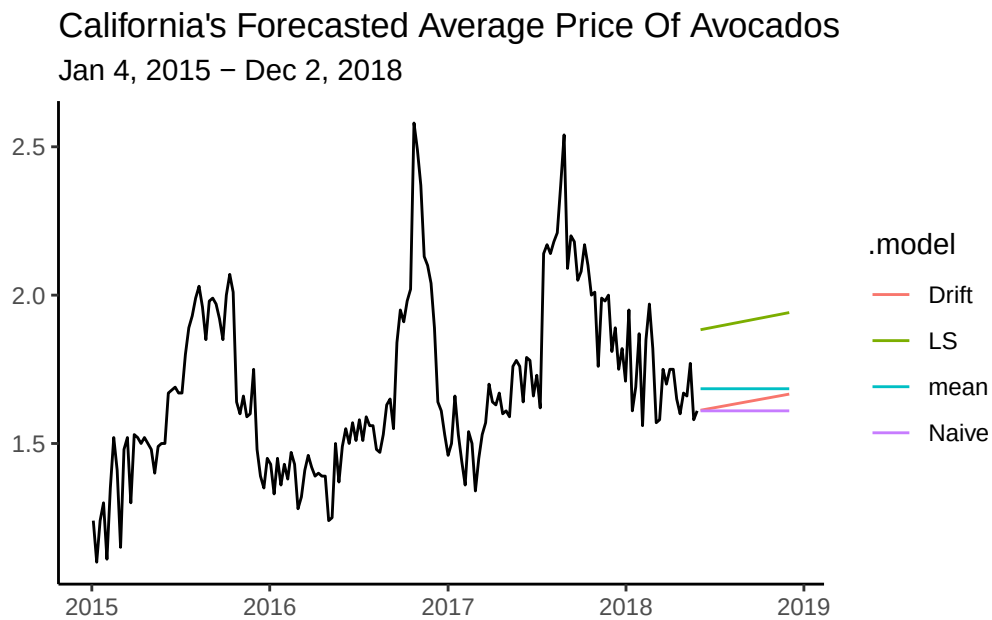
Root Mean Squared Error (RMSE) - square root of MSE.

```
accuracy(fit) %>%
  gt() %>%
  cols_align("center") %>%
  tab_header(title = md("**Model Fit**")) %>%
  tab_style(locations = cells_column_labels(columns = everything()),
    style = list(cell_borders(sides = "bottom", weight = px(3)),
      cell_text(weight = "bold"))) %>%
  fmt_number(columns = c(ME, RMSE, MAE, MPE, MAPE, MASE, RMSSE, ACF1),
    decimals = 2) %>%
  tab_style_body(
    style = cell_fill(color = "lightgreen"),
    values = "Naive",
    targets = "row")
```

Forecast

```
# Forecast series using forecast(). Specify number of periods (h) as 27 weeks.
calits_fc <- fit %>% forecast(h=27)
```

```
# Create graph with the forecast and training set using autoplot() and autolayer() functions
# Set argument level null to omit prediction intervals.
calits_fc %>% autoplot(level=NULL) + theme_classic() +
  autolayer(calits_train, average_price) +
  labs(y="", title= "California's Forecasted Average Price Of Avocados",
       subtitle = "Jan 4, 2015 - Dec 2, 2018",
       x="")
```



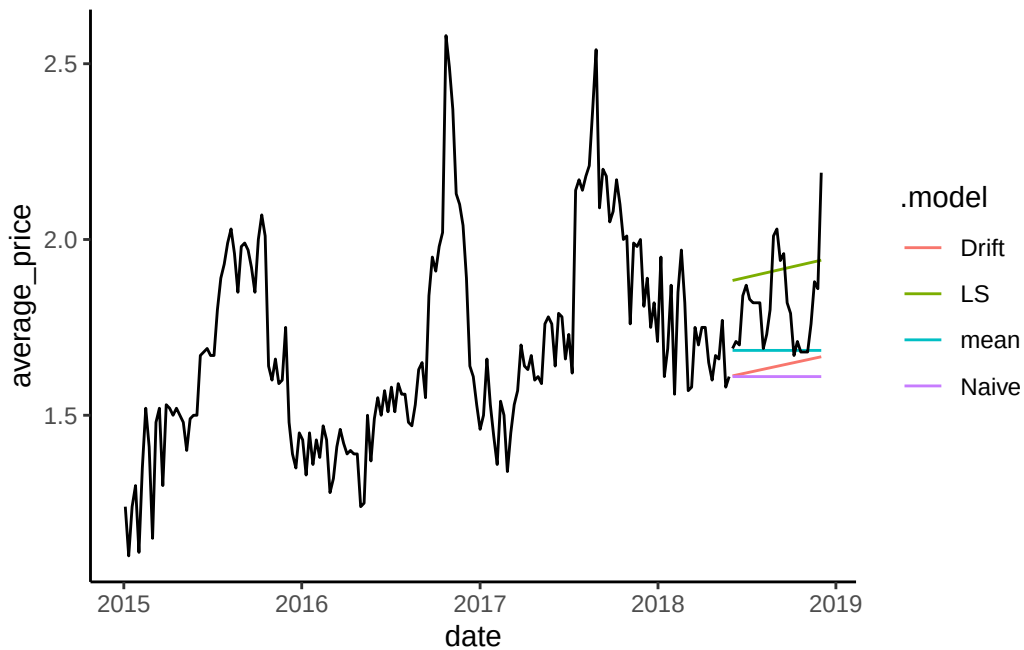
Overfitting

Occurs when the model is too flexible. This is why we split data into a training and test set.

```
# Create training and test sets.
cali %>%
  as_tsibble(key=geography,
             index=date, regular=T) %>%
  filter_index("2018-06-02"~"2018-12-02") -> calits_test

cali %>% as_tsibble(index=date) %>%
  filter_index(.~"2018-12-02") -> calits
```

```
# Plot the training set, forecast, and test set.
calits_fc %>% autoplot(level=NULL) +
  theme_classic() +
  autolayer(calits_train, average_price) +
  autolayer(calits_test, average_price)
```



```
# Evaluate accuracy.
accuracy(calits_fc, calits_test) %>% select(-ACF1, RMSSE, MASE)
```

```
# A tibble: 4 x 9
```

	.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE
	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>	<dbl>
1	Drift	Test	0.175	0.214	0.175	9.24	9.24	NaN	NaN
2	LS	Test	-0.0983	0.158	0.138	-5.88	7.78	NaN	NaN
3	Naive	Test	0.204	0.240	0.204	10.8	10.8	NaN	NaN
4	mean	Test	0.130	0.180	0.132	6.72	6.85	NaN	NaN

Cross Validation

Creating several training and test sets.

Model Fit Cross Validation

.model	.type	ME	RMSE	MAE	MPE	MAPE	MASE	RMSSE	ACF1
Drift	Test	0.00	0.14	0.10	-0.58	6.09	1.01	1.01	-0.21
LS	Test	-0.05	0.27	0.21	-4.64	12.35	2.05	1.94	0.85
Mean	Test	0.12	0.29	0.21	5.36	11.73	2.12	2.07	0.85
Naive	Test	0.00	0.14	0.10	-0.15	5.96	0.99	1.00	-0.21

```
# Set first three observations as training set. Estimate model and forecast fourth observation.
# Forecast error is recorded and training set is adjusted to forecast nth observation.
avocado_cv <- calits_train %>%
  stretch_tsibble(.init = 3, .step = 1)
```

```
avocado_cv %>%
  model(Mean=MEAN(average_price),
        Naive=RW(average_price),
        Drift=RW(average_price ~ drift()),
        LS=TSLM(average_price~date)) %>%
  forecast(h = 1) %>% accuracy(calits) %>%
  gt() %>%
  cols_align("center") %>%
  tab_header(title = md("**Model Fit Cross Validation**")) %>%
  tab_style(locations = cells_column_labels(columns = everything()),
            style = list(cell_borders(sides = "bottom", weight = px(3)),
                          cell_text(weight = "bold"))) %>%
  fmt_number(columns = c(ME, RMSE, MAE, MPE, MAPE, MASE, RMSSE, ACF1),
             decimals = 2) %>%
  tab_style_body(
    style = cell_fill(color="lightgreen"),
    values = "Naive",
    targets = "row")
```

Other Accuracy Measures

```
calits_train %>%
  model(LS=TSLM(average_price~trend()),
        ETS=ETS(average_price),
        ARIMA=ARIMA(average_price))%>%
```

```
glance() %>%
  select('.model', "AIC", "AICc", "BIC")
```

Warning: 1 error encountered for ARIMA

[1] The `urca` package must be installed to use this functionality. It can be installed with

```
# A tibble: 2 x 4
  .model    AIC  AICc   BIC
  <chr>   <dbl> <dbl> <dbl>
1 LS     -476. -476. -467.
2 ETS     203.  203.  213.
```

Practice Exercises

Question 1

You've been tasked by the Japanese government with an important mission: forecast the country's population by 2030. Your projections will help them effectively allocate resources and plan for future demands. To accomplish this, you'll rely on population data available in the UNPOP file.

Build two models using the TSLM approach. One with a simple linear trend and another with a quadratic trend (use $I(\text{trend()}^2)$ in R to incorporate a quadratic term). Compare the models by calculating their BIC (Bayesian Information Criterion), which will help you identify which model is more accurate. Provide your population estimate for 2030 using the model with the lower BIC. Based on your findings, determine whether Japan's population is expected to increase or decrease by 2030.

```
# Load data
data<-read_csv("https://jagelves.github.io/Data/UNPOP.csv")
```

Warning: One or more parsing issues, call `problems()` on your data frame for details, e.g.:

```
dat <- vroom(...)
problems(dat)
```

Rows: 21983 Columns: 65

-- Column specification -----

Delimiter: ","

chr (7): Variant, Area, Notes, IS03, IS02, Type, Population_Annual_Doubling...

dbl (58): Index, Location_code, SDMX_code, Parent code, Year, Total_Populati...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
# Create tsibble. Filter Japan, select year, population, and ISO3 columns. Set index to year
data %>% filter(ISO3=="JPN") %>%
  select(Year, Total_Population, ISO3) %>%
  as_tsibble(index=Year, key=ISO3) -> JAP_ts

# Create models
JAP_ts %>% model(TSLML=TSLM(Total_Population~trend()),
               TSLMQ=TSLM(Total_Population~trend()+I(trend()^2))) -> fit

# Compare models using BIC.
fit %>% glance() %>%
  select('.model', "AIC", "AICc", "BIC")
```

```
# A tibble: 2 x 4
  .model    AIC  AICc    BIC
  <chr>    <dbl> <dbl> <dbl>
1 TSLML  1270. 1271. 1277.
2 TSLMQ  1048. 1048. 1057.
```

```
# Forecast
fit %>% forecast(h=7) %>% filter(.model=="TSLMQ")
```

```
# A tibble: 7 x 5 [1Y]
# Key:      ISO3, .model [1]
  ISO3  .model  Year
  <chr> <chr>   <dbl>
1 JPN   TSLMQ    2024
2 JPN   TSLMQ    2025
3 JPN   TSLMQ    2026
4 JPN   TSLMQ    2027
5 JPN   TSLMQ    2028
6 JPN   TSLMQ    2029
7 JPN   TSLMQ    2030
# i 2 more variables: Total_Population <dist>, .mean <dbl>
```

Population is expected to decrease by roughly 123,000 people.

Question 2

Fertility rates are crucial because they directly impact population growth and demographic structure. Low fertility rates can lead to an aging population, shrinking workforce, and challenges in sustaining economic growth and social welfare programs. To maintain a stable population in the U.S., the total fertility rate (TFR) needs to be around 2.1 children per woman (i.e. the replacement rate). If the TFR falls below this level for an extended period, the population may decline without immigration. Use the data found in the UNPOP file to plot the fertility rate for the US from 1960-2023 and the current replacement rate. What is the two year projected fertility rate when using a simple linear trend? How does this change when running the same model with a log transformation on the fertility variable?

```
library(ggthemes)
```

Warning: package 'ggthemes' was built under R version 4.4.2

```
data<-read_csv("https://jagelves.github.io/Data/UNPOP.csv")
```

Warning: One or more parsing issues, call `problems()` on your data frame for details, e.g.:

```
dat <- vroom(...)
problems(dat)
```

Rows: 21983 Columns: 65

-- Column specification -----

Delimiter: ","

chr (7): Variant, Area, Notes, IS03, IS02, Type, Population_Annual_Doubling...

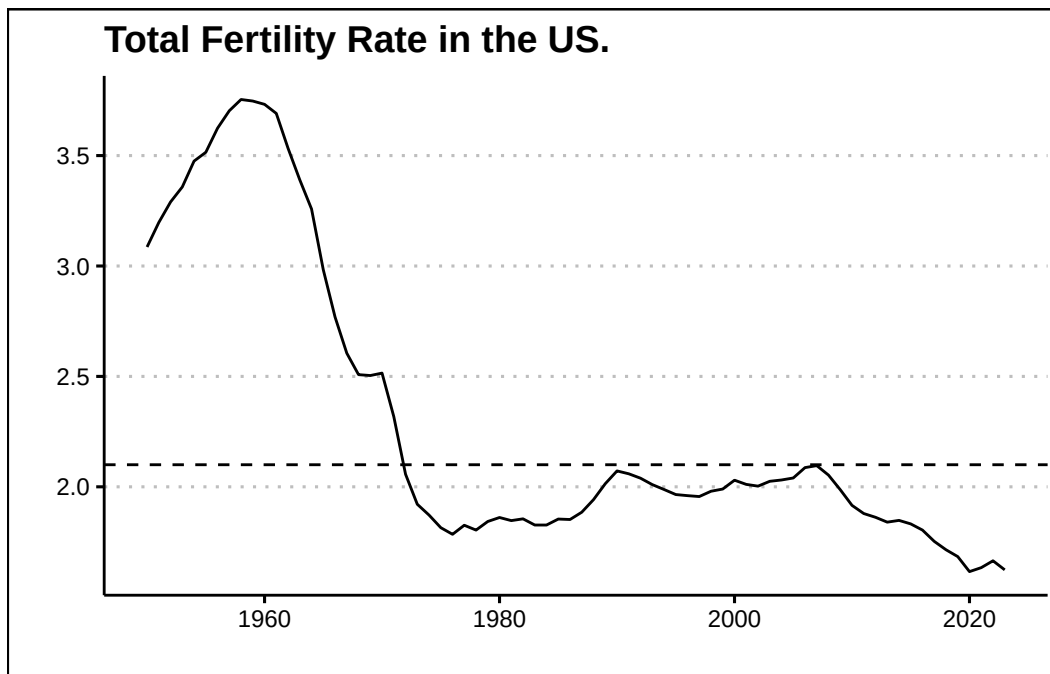
dbl (58): Index, Location_code, SDMX_code, Parent code, Year, Total_Populati...

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
data %>% filter(IS03=="USA") %>%
  select(Year, Total_Fertility_Rate,IS03) %>%
  as_tsibble(index=Year,key=IS03) -> USFert
```

```
USFert %>%
  autoplot(.vars=Total_Fertility_Rate) + theme_clean() +
  geom_hline(yintercept = 2.1, lty=2) +
  labs(title="Total Fertility Rate in the US.",
       x="", y="")
```

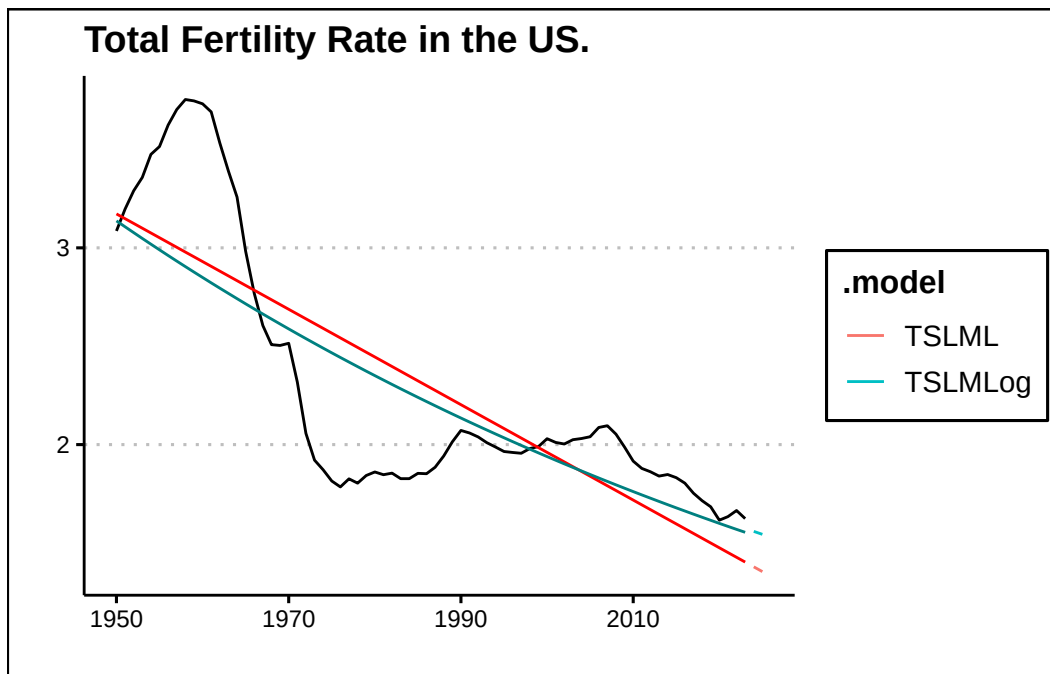


```
# Linear trend shows fertility rate will decrease to 1.35. Log transformation shows a slower
USFert %>% model(TSLML=TSLM(Total_Fertility_Rate~trend()),
                TSLMLog=TSLM(log(Total_Fertility_Rate)~trend())) -> fit

fit %>% forecast(h=2)
```

```
# A fable: 4 x 5 [1Y]
# Key:      IS03, .model [2]
  IS03  .model  Year
  <chr> <chr>   <dbl>
1 USA   TSLML    2024
2 USA   TSLML    2025
3 USA   TSLMLog  2024
4 USA   TSLMLog  2025
# i 2 more variables: Total_Fertility_Rate <dist>, .mean <dbl>
```

```
fit %>% forecast(h=2) %>% autoplot(level=NULL) +
  autolayer(USFert,Total_Fertility_Rate) + theme_clean() +
  autolayer(fit %>% augment() %>% filter(.model=="TSLML"),.fitted, col="red") +
  autolayer(fit %>% augment() %>% filter(.model=="TSLMLog"),.fitted, col="#008080") +
  labs(title="Total Fertility Rate in the US.",
       x="", y="")
```



Question 3

Refer back to Problem 2, where your task is to forecast the fertility rate. For both the linear and logarithmic models you've developed, calculate the RMSE (Root Mean Squared Error) using the models' fitted values. Use the `augment()` function to extract the residuals for each model, then square these residuals, average them, and finally take the square root. Based on the RMSE, determine which model provides a better fit for the data.

```
# RMSE for linear model
fit %>% augment() %>% filter(.model=="TSLML") %>%
  pull(.resid) %>% ^\` (2) %>% mean() %>% sqrt()
```

```
[1] 0.4055513
```

```
# RMSE for log model
fit %>% augment() %>% filter(.model=="TSLMLog") %>%
  pull(.resid) %>% ^\` (2) %>% mean() %>% sqrt()
```

```
[1] 0.3850176
```

Log has lower RMSE; therefore, fits the model better.

Question 4

Refer back to Problem 2, where your task is to forecast the fertility rate. For both the linear and logarithmic models you've developed, perform leave-one-out cross-validation. Initialize the cross-validation with eight observations and use a step of one. Which model performs better according to the RMSE?

```
USFert %>%
  stretch_tsibble(.init = 8, .step = 1) %>%
  model(TSLML=TSLM(Total_Fertility_Rate~trend()),
        TSLMLog=TSLM(log(Total_Fertility_Rate)~trend())) %>% forecast(h=1) %>% accuracy(USFert)
select(.model,RMSE)
```

Warning: The future dataset is incomplete, incomplete out-of-sample data will be treated as missing
1 observation is missing at 2024

```
# A tibble: 2 x 2
  .model    RMSE
  <chr>    <dbl>
1 TSLML    0.457
2 TSLMLog  0.359
```

TSLMLog performs better.

Question 5

Grocers Inc. purchases avocados at Richmond/Norfolk's market price and sells them in their stores for \$1.5 each. Using the Avocado 2020-2023 [Excel] price data file, forecast the margin for the company over the next 4 weeks using a simple mean, the NAIVE, and simple trend. Use only data from W2 2023 onward.

```
avo<-read_csv("https://jagelvels.github.io/Data/avocado2020-2023.csv")
```

```
Rows: 21240 Columns: 10
```

```
-- Column specification -----
```

```
Delimiter: ","
```

```
chr (4): Geography, Timeframe, Date, Type
```

```
dbl (6): AveragePrice, TotalUnits, 4046Units, 4225Units, 4770Units, TotalBag...
```

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
avo %>% select(Geography, Date, Type, AveragePrice) %>%
  filter(Geography=="Richmond/Norfolk") %>%
  filter(Type=="Conventional") %>%
  mutate(Date=mdy(Date)) %>%
  mutate(Date=yearweek(Date)) %>%
  select(Date, AveragePrice) %>%
  distinct(Date, .keep_all = TRUE) %>%
  arrange(Date) %>%
  as_tsibble(index=Date) %>%
  filter_index("2023 W2"~.) -> avo_ts
```

```
avo_ts %>% model(MEAN=MEAN(AveragePrice),
               NAIVE=NAIVE(AveragePrice),
               TSLM=TSLM(AveragePrice~trend())) -> fit
```

```
fit %>% forecast(h=4) %>%
  as_tibble() %>%
  select(Date, .model, .mean) %>%
  mutate(Margin=(1.5-.mean)/1.5)
```

A tibble: 12 x 4

	Date	.model	.mean	Margin
	<week>	<chr>	<dbl>	<dbl>
1	2023 W25	MEAN	1.03	0.312
2	2023 W26	MEAN	1.03	0.312
3	2023 W27	MEAN	1.03	0.312
4	2023 W28	MEAN	1.03	0.312
5	2023 W25	NAIVE	1.13	0.247
6	2023 W26	NAIVE	1.13	0.247
7	2023 W27	NAIVE	1.13	0.247
8	2023 W28	NAIVE	1.13	0.247
9	2023 W25	TSLM	1.09	0.272
10	2023 W26	TSLM	1.10	0.268
11	2023 W27	TSLM	1.10	0.265
12	2023 W28	TSLM	1.11	0.262